SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security. (BL3, BL6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

Coding Challenge

Code of Conduct:

I **KESHAVARAM L/192511253**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1: Secure Email Transmission using S/MIME

Implementation: hybrid encryption (RSA-OAEP wraps a random AES-256-GCM session key) for confidentiality, combined with an RSA-PSS digital signature over the plaintext for authentication — mirroring how real S/MIME messages are constructed (sign-then-encrypt, hybrid key transport):

```
def smime_send(message, sender_priv, recipient_pub):
    signature = sender_priv.sign(message, padding.PSS(...), hashes.SHA256())
    session_key = AESGCM.generate_key(bit_length=256)
    ciphertext = AESGCM(session_key).encrypt(nonce, message + b"||SIG||" + signature, None)
    enc_session_key = recipient_pub.encrypt(session_key, padding.OAEP(...))   # RSA key transport
    return {"enc_session_key": enc_session_key, "nonce": nonce, "ciphertext": ciphertext}

def smime_receive(envelope, recipient_priv, sender_pub):
    session_key = recipient_priv.decrypt(envelope["enc_session_key"], padding.OAEP(...))
    payload = AESGCM(session_key).decrypt(envelope["nonce"], envelope["ciphertext"], None)
    message, signature = payload.split(b"||SIG||")
    sender_pub.verify(signature, message, padding.PSS(...), hashes.SHA256())   # authentication check
```

Test results

```
Original message:   b'Board approval: proceed with the Q3 acquisition at the agreed terms.'
Recovered message:  b'Board approval: proceed with the Q3 acquisition at the agreed terms.'
Message intact?     True
Signature valid?    True

Tampering with the ciphertext in transit:
  Tampering detected -> decryption rejected: InvalidTag

An eavesdropper without the recipient's private key attempting to decrypt:
  Decryption failed as expected -> ValueError
```

Analysis: how confidentiality and authentication are ensured

Confidentiality: the message is encrypted with a one-time AES-256 session key, and that session key is itself encrypted (RSA-OAEP) so only the holder of the recipient's private key can recover it and, in turn, the message. As demonstrated, a third party without that private key cannot decrypt the envelope at all.

Authentication: the RSA-PSS signature is computed with the sender's private key over the original message, so only the real sender could have produced it; the recipient verifies it with the sender's public key. Because AES-GCM also provides its own authentication tag, any tampering with the ciphertext in transit is independently detected and rejected before the message is even reached, as shown above.

Question 2: PGP-based File Encryption and Decryption

Implementation: RSA key generation, PEM export to simulate public-key distribution, PGP-style compress-then-encrypt with a hybrid RSA/AES scheme, and a digital signature for authentication:

```
def pgp_encrypt_file(file_bytes, sender_priv, recipient_pub_pem):
    recipient_pub = serialization.load_pem_public_key(recipient_pub_pem)
    compressed = zlib.compress(file_bytes)                        # PGP compresses before encrypting
    session_key = AESGCM.generate_key(bit_length=256)
    ciphertext = AESGCM(session_key).encrypt(nonce, compressed, None)
    enc_session_key = recipient_pub.encrypt(session_key, padding.OAEP(...))
    signature = sender_priv.sign(file_bytes, padding.PSS(...), hashes.SHA256())  # authentication
    return {"enc_session_key": ..., "nonce": ..., "ciphertext": ciphertext, "signature": signature}
```

Test results

```
Bob's distributed public key (PEM, first 60 chars): b'-----BEGIN PUBLIC KEY-----...' (simulated key
distribution)

Original file size:  17028 bytes
Compressed+encrypted size: 154 bytes (0.9% of original, thanks to compression)
File recovered correctly? True
Signature (authenticity) verified? True

--- Why hybrid encryption is necessary/efficient ---
Direct RSA encryption of the full (17028-byte) file FAILS: Encryption failed
Hybrid (PGP-style) encryption of the same file took 0.69 ms
(RSA only wraps a 32-byte session key; AES handles the bulk data)
```

Evaluation: security and efficiency

RSA cannot encrypt data larger than its key size at all (confirmed above — a direct attempt on the full file fails outright), so pure-RSA file encryption is not just slow but often impossible for realistic file sizes. PGP's hybrid design uses RSA only to wrap a small, fixed-size AES session key, while the bulk file data is encrypted with fast symmetric AES — this is what makes the whole operation complete in under a millisecond even though a 2048-bit RSA operation alone can take several milliseconds. Compressing before encryption additionally reduces the data actually transmitted (dramatically so for repetitive content, as shown), and the digital signature layered on top ensures the recipient can confirm the file truly came from the claimed sender and was not altered — giving PGP both strong security (confidentiality + authenticity) and practical efficiency at the same time.

Question 3: IPSec Tunnel Mode Simulation

Implementation: AES-GCM used as the ESP encryption engine, applied two different ways to demonstrate Transport Mode (payload only) versus Tunnel Mode (entire original packet, re-encapsulated under new gateway addressing):

```
def esp_transport_mode(packet, key):
    enc_payload = AESGCM(key).encrypt(nonce, packet["payload"].encode(), None)
    return {"src": packet["src"], "dst": packet["dst"], "ttl": packet["ttl"],  # visible
            "esp_nonce": nonce, "esp_payload": enc_payload}                    # only payload encrypted

def esp_tunnel_mode(inner_packet, key, outer_src, outer_dst):
    serialized_inner = json.dumps(inner_packet).encode()      # ENTIRE original packet, header included
    enc_inner = AESGCM(key).encrypt(nonce, serialized_inner, None)
    return {"src": outer_src, "dst": outer_dst, "ttl": 64,     # brand-new outer header
            "esp_nonce": nonce, "esp_payload": enc_inner}
```

Test results

```
=== TRANSPORT MODE ===
What an observer on the network sees (headers in the clear):
  src=10.0.1.15  dst=10.0.2.30  ttl=64  payload=<encrypted, 45 bytes>
Recovered payload after decryption: GET /payroll/records HTTP/1.1

=== TUNNEL MODE ===
What an observer on the network sees (only the new GATEWAY addresses):
  src=203.0.113.1  dst=203.0.113.2  ttl=64  payload=<encrypted, 111 bytes, entire original packet
inside>
  (the ORIGINAL src=10.0.1.15 / dst=10.0.2.30 / payload are completely hidden)
Recovered original packet after decryption at the far-end gateway:
```

```
  {'src': '10.0.1.15', 'dst': '10.0.2.30', 'ttl': 64, 'payload': 'GET /payroll/records HTTP/1.1'}

=== Key difference confirmed ===
Transport mode leaks the real endpoints: 10.0.1.15 -> 10.0.2.30
Tunnel mode only exposes the gateways:    203.0.113.1 -> 203.0.113.2
```

Analysis: Tunnel Mode vs. Transport Mode

Transport Mode only encrypts the payload of the original IP packet, leaving the original source and destination addresses fully visible — appropriate for direct end-to-end host-to-host communication where routing devices in between still need to see the real IP header, but it discloses who is talking to whom. Tunnel Mode, as demonstrated, encrypts the entire original packet (including its header) and wraps it inside a completely new outer packet addressed between two gateways/security endpoints; any observer on the path between the gateways sees only the gateway-to-gateway traffic and cannot determine the real internal source or destination at all. This is why Tunnel Mode is the standard choice for site-to-site VPNs (protecting internal network topology), while Transport Mode is typically used for direct host-to-host protection where hiding the endpoints themselves is not required.

Question 4: SSL/TLS Handshake Simulation

Implementation: CA-issued RSA certificate for the server, ephemeral ECDHE (elliptic-curve Diffie-Hellman) key exchange with the ephemeral public key signed by the server's certified long-term key (binding the ephemeral exchange to a verified identity), HKDF session-key derivation, and AES-GCM secure data transfer:

```
def tls_handshake(server_cert, server_priv, ca_pub):
    if not verify_certificate(server_cert, ca_pub):                  # 1) certificate exchange &
validation
        raise ValueError("Certificate validation failed")
    client_eph_priv = ec.generate_private_key(ec.SECP256R1())        # 2) ephemeral EC key pairs
    server_eph_priv = ec.generate_private_key(ec.SECP256R1())
    server_priv.sign(pub_bytes(server_eph_pub), padding.PSS(...))    # bind ephemeral key to identity
    shared = client_eph_priv.exchange(ec.ECDH(), server_eph_pub)     # 3) ECDH shared secret
    session_key = HKDF(algorithm=hashes.SHA256(), length=32, ...).derive(shared)  # 4) derive session
key
    return session_key
```

Test results

```
Handshake complete in 1.41 ms; derived session key: ffe2362b05afa66683a1644374640c3a...
Client request sent (encrypted): b'\x87\x82\xe3V\xd0t\x19\xe0\xb7\xd3...' ...
Server received & decrypted:     b'GET /account/balance HTTP/1.1\r\nHost: secure.example.com\r\n'
Data transferred intact?          True

Attempting handshake with a forged (non-CA-signed) certificate:
  Handshake correctly aborted: Certificate validation failed -- aborting handshake

Two independent handshakes with the SAME long-term server certificate:
Session key #1: ffe2362b05afa66683a16443 ...
Session key #2: ef3137945cfc236504d9efab ...
Are they different? True (ephemeral keys mean each session is independently secure -- forward secrecy)
```

Evaluation: how the handshake ensures secure web communication

- Authentication: the server's certificate is validated against a trusted CA before any key material is trusted, and a forged/self-signed certificate is correctly rejected, as shown — preventing impersonation.

- Confidentiality and integrity: the session key derived via ECDHE + HKDF is used with AES-GCM, which both encrypts the data and authenticates it (tamper detection built in), so the client's request arrives at the server completely unreadable to any eavesdropper yet perfectly intact for the legitimate recipient.
- Forward secrecy: because a fresh ephemeral EC key pair is generated for every handshake, two sessions with the exact same long-term certificate produce completely different session keys (confirmed above) — so even if the server's long-term private key were later compromised, past recorded sessions cannot be decrypted retroactively, which is precisely why modern TLS defaults to ECDHE over static RSA key exchange.

Question 5: Email Phishing Detection Tool

Implementation: feature extraction from email subject, sender header, and embedded URL, followed by a logistic-regression classifier trained on a labeled dataset of phishing vs. legitimate emails:

```
def extract_features(subject, sender, url):
    features = {}
    features["urgency_word_count"] = sum(w in subject.lower() for w in URGENCY_WORDS)
    features["sender_domain_has_hyphen"] = int("-" in sender_domain)
    features["sender_looks_like_brand_impersonation"] = int(...)      # header feature
    features["url_has_raw_ip"] = int(bool(re.search(r"https?://\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}",
url)))
    features["url_is_shortener"] = int(any(s in url for s in ["bit.ly", "tinyurl", "goo.gl"]))
    features["url_is_http_not_https"] = int(url.startswith("http://"))
    features["url_suspicious_tld"] = int(any(...))                    # URL pattern features
    return features

clf = LogisticRegression(max_iter=1000).fit(X_train, y_train)
```

Test results

```
Test set size: 6  (phishing=3, legitimate=3)
Accuracy:  1.00
Precision: 1.00
Recall:    1.00
F1 score:  1.00
Confusion matrix [[TN, FP],[FN, TP]]:
 [[3 0]
 [0 3]]

Learned feature weights (higher = stronger phishing signal):
  sender_domain_len                      +1.050
  sender_domain_has_hyphen               +0.294
  url_is_http_not_https                  +0.294
  urgency_word_count                     +0.285
  url_has_raw_ip                         +0.157
  sender_looks_like_brand_impersonation  +0.137

Predictions on new, unseen emails:
  'Your account will be locked, verify now!...' -> predicted=PHISHING (true=PHISHING, probability=1.00)
  'Project kickoff notes from this morning...'  -> predicted=legitimate (true=legitimate,
probability=0.07)
```

Analysis: effectiveness of the model in identifying malicious emails

On this small, clearly-separated dataset the classifier achieves perfect accuracy/precision/recall, and it correctly generalizes to two completely unseen emails, correctly flagging a new phishing attempt with 100% confidence and a new legitimate email with only 7% phishing probability. The learned feature weights are also interpretable and

consistent with security intuition — raw-IP URLs, HTTP-instead-of-HTTPS links, hyphenated look-alike sender domains, and urgency language all push the prediction toward 'phishing', matching how real phishing emails are typically constructed.

This result should be read with appropriate caution: the dataset is small and synthetic, so this evaluation demonstrates that the feature-extraction approach is sound in principle rather than proving production-level accuracy. A deployed system would need a much larger, real-world labeled dataset; would need to handle adversarial evasion (attackers deliberately avoiding known trigger words or using freshly-registered but plausible-looking domains); would need to guard against false positives on legitimate marketing/transactional email (which can also contain urgency language and shortened links); and would benefit from richer features such as sender authentication results (SPF/DKIM/DMARC), attachment analysis, and periodic retraining as phishing tactics evolve.